

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 06 -

BOAS PRÁTICAS, SEGURANÇA E TROUBLESHOOTING

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMANDA

O QUE VEM POR AÍ?

Depois de entender como criar e orquestrar containers com Docker e Docker Compose, chega o momento de encarar o que realmente separa quem só “faz rodar” de quem entrega soluções robustas: manter tudo rodando bem, com segurança e clareza no diagnóstico. Nesta última aula, vamos conversar sobre boas práticas que ajudam a evitar “dor de cabeça” lá na frente, explorar medidas fundamentais de segurança que todo container deveria seguir, e aprender como identificar e resolver problemas quando algo dá errado — porque uma hora, vai dar. Se você quer levar seu ambiente Dockerizado para o próximo nível, esta aula é para você.

HANDS ON

Chegou a hora de colocar a “mão na massa” e testar tudo o que aprenderemos sobre boas práticas, segurança e troubleshooting com Docker. Nesta etapa, vamos simular situações reais de uso e mostrar como aplicar estratégias que deixam seu ambiente mais leve, mais seguro e muito mais confiável no dia a dia. Nada de exemplos soltos — o foco aqui é mostrar como esses conceitos aparecem quando você realmente precisa lidar com containers em produção.

Começaremos **otimizando a criação de imagens e containers**: como deixar tudo mais enxuto e eficiente, sem abrir mão da legibilidade e da manutenção do projeto. Em seguida, mergulharemos no tema **segurança**, com dicas e práticas para reduzir riscos, evitar armadilhas e garantir que seus containers estejam bem protegidos — desde o build até a execução. Por fim, enfrentaremos situações comuns de erro e lentidão, e mostraremos como investigar o que está acontecendo usando comandos e ferramentas que todo mundo deveria ter na manga.

Além disso, vamos configurar usuários não-root, verificar vulnerabilidades em imagens e interpretar erros comuns de inicialização. Tudo com contexto e aplicação direta, para que você entenda não só o como, mas principalmente o porquê de cada escolha. Se você quer fechar essa jornada com Docker sentindo segurança para criar, depurar e proteger seus containers, esse é o momento! Vamos nessa!

SAIBA MAIS

Uso de `.dockerignore`: economize tempo e evite vazamentos

Se você está usando Docker para empacotar aplicações, precisa conhecer o `.dockerignore`. Ele funciona de forma parecida com o `.gitignore`, e serve para excluir arquivos e pastas do contexto de build da imagem. Parece simples, mas isso impacta diretamente na **performance do build**, na **segurança da imagem gerada** e também na **prevenção de bugs difíceis de rastrear**.

Por que ele é importante?

Toda vez que você roda um `docker build`, o Docker precisa copiar o **contexto de build** (ou seja, todos os arquivos e pastas do diretório atual) para dentro do processo de build, e isso inclui coisas que você não deveria mandar para dentro da imagem: arquivos de configuração locais, dependências, secretkeys, diretórios de cache, e por aí vai.

Sem um `.dockerignore` bem configurado, você pode:

- Subir acidentalmente arquivos sensíveis (como `.env`, `id_rsa`, `.git/config`);
- Enviar megabytes (ou gigabytes) de arquivos desnecessários para o build;
- Quebrar a camada de cache, fazendo builds demorarem mais do que o necessário.

Exemplo prático

Imagine que seu projeto tem a seguinte estrutura:

```
meu-app/  
├── node_modules/  
├── .env  
├── .git/  
├── build/  
├── src/  
├── Dockerfile  
└── docker-compose.yml
```

Código-fonte 1 – Estrutura de um projeto fictício
Fonte: elaborado pelo autor (2025)

Um bom `.dockerignore` seria:

```
node_modules
.env
.git
build
*.log
```

Código-fonte 2 – Exemplo de um bom `.dockerignore`
Fonte: elaborado pelo autor (2025)

Esse simples arquivo já previne:

- Que você mande milhares de arquivos do `node_modules` para o interior da imagem;
- Que o `.env` vá para dentro do container em produção;
- Que a pasta `.git` com histórico e metadados seja copiada inutilmente;
- Que arquivos de build anteriores entrem por engano na nova imagem.

Dica extra

Se você usa o GitHub Copilot ou outros assistentes de código, sempre revise se o `.dockerignore` sugerido cobre os arquivos do seu projeto. Cada stack tem necessidades diferentes e um bom `.dockerignore` deve ser escrito com carinho pra sua realidade.

Ah, e uma boa prática: mantenha seu `.dockerignore` próximo do seu Dockerfile, na raiz do projeto. Assim fica mais fácil de entender o que está sendo levado para dentro da imagem e o que fica fora.

Logs estruturados e centralização com Loki, Fluentd e Promtail

Quando se trabalha com containers, especialmente em ambientes mais robustos e com múltiplas instâncias, confiar apenas no `docker logs` deixa de ser suficiente. Isso porque os logs ficam fragmentados entre vários containers e, caso um container seja destruído, os logs geralmente se perdem. A solução? Centralizar tudo e garantir que os logs sejam estruturados.

Logs estruturados nada mais são do que registros em formato padronizado (geralmente JSON), onde cada entrada tem campos consistentes como `timestamp`, `level`, `message`, `service`, entre outros. Isso facilita não só a leitura, mas principalmente a busca e a correlação de eventos.

E é aqui que entram ferramentas como o **Loki**, o **Fluentd** e o **Promtail**:

- **Promtail** é um agente que roda junto dos seus containers, lê os logs locais e os envia para o Loki;
- **Loki** é o sistema de back-end, desenvolvido pela Grafana Labs, especializado em armazenar logs de forma eficiente, com um modelo similar ao Prometheus (mas para logs);
- **Fluentd** é uma alternativa poderosa para coleta, processamento e roteamento de logs. Ele pode substituir o Promtail em cenários mais complexos, com necessidade de transformar ou enriquecer os logs antes do envio.

A grande vantagem desse trio é a capacidade de coletar logs de múltiplos containers, normalizá-los e enviá-los para uma interface como o Grafana, onde você pode fazer queries detalhadas, montar dashboards e correlacionar eventos com métricas.

Se você está começando agora, vale montar um setup simples com Docker Compose que inclua Promtail, Loki e Grafana. Além de ser uma ótima experiência prática, você vai entender o valor de logs estruturados de verdade.

Dica: Combine logs estruturados com alertas baseados em eventos críticos. Isso eleva a sua aplicação para um novo nível de observabilidade.

Docker Events: monitore em tempo real o que acontece nos containers

Saber o que está rolando em tempo real dentro do seu ambiente Docker pode ser a diferença entre detectar um problema antes que ele escale ou passar horas tentando descobrir o que aconteceu. O comando `dockerevents` é uma ferramenta nativa, leve e extremamente útil para acompanhar as mudanças de estado dos containers, imagens, volumes e outros objetos Docker.

Com ele, você pode visualizar eventos como criação, inicialização, parada, remoção, attach, kill e até falhas de rede ou reinicializações. Tudo isso com timestamps e informações precisas.

Por exemplo, se você quiser monitorar os eventos de todos os containers:

```
dockerevents
```

Comando de Prompt 1 – Monitorando eventos de todos os containers

Fonte: elaborado pelo autor (2025)

Se quiser filtrar por container específico:

```
docker events --filter container=meu-container
```

Comando de Prompt 2 – Filtrando containers específicos

Fonte: elaborado pelo autor (2025)

Ou ainda, se quiser acompanhar apenas eventos de tipo específico (por exemplo, apenas eventos de tipo **container**):

```
dockerevents --filter type=container
```

Comando de Prompt 3 – Acompanhando eventos de tipo específico

Fonte: elaborado pelo autor (2025)

Esse comando é especialmente útil quando estamos fazendo troubleshooting em tempo real, testando algum script de automação ou tentando entender o comportamento da infraestrutura durante deploys.

Quer integrar isso com uma ferramenta de observabilidade? Dá para usar o output do **dockerevents** junto com scripts em Shell, Python ou até encaminhar para um sistema de logs estruturado.

Fica aqui um convite: na próxima vez que algo estranho acontecer no seu ambiente Docker, abra um terminal e roda um **dockerevents**. Você pode se surpreender com o que vai descobrir rodando “por trás dos panos”.

Diagnóstico com dockerinspect: lendo metadados como um(a) detetive

Se você já usou o **dockerinspect**, provavelmente foi para buscar um IP ou o path de um volume. Mas, o que pouca gente explora, é o poder de investigação que esse comando oferece para diagnósticos mais profundos e bem informados.

O **dockerinspect** devolve uma saída JSON detalhada sobre containers, imagens, volumes, redes... praticamente qualquer recurso do Docker. Com ele, é possível saber:

- Qual comando original está sendo executado no container (**Config.Cmd**), se o container está utilizando um volume e onde ele está montado (**Mounts**);

- Quais portas estão expostas e mapeadas ([NetworkSettings.Ports](#));
- Qual é a imagem base ([Image](#));
- Se o container está sendo reiniciado frequentemente ou se está parado por erro ([State](#));

Quer saber se o container está morrendo por falta de memória? Verifique [State.OOMKilled](#). Quer automatizar uma checagem? Use o `--format` para filtrar exatamente o que precisa:

```
dockerinspect --format='{{.State.Status}}' meu_container
```

Comando de Prompt 4 – Usando `--format`
Fonte: elaborado pelo autor (2025)

Esse comando vai te devolver apenas o status atual, como `running` ou `exited`, sem poluir o seu terminal. E mais, combinar `dockerinspect` com ferramentas como `jq` ou usar em scripts de shell pode turbinar suas rotinas de troubleshooting.

Prático, poderoso e já incluído na sua instalação do Docker. Da próxima vez que algo der errado, não chute — inspecione!

Checklist de produção: o que revisar antes de subir o seu container

Antes de colocar qualquer container em produção, vale a pena fazer uma última checagem. A ideia aqui não é só garantir que o container *roda*, mas sim que ele está pronto para um ambiente real: seguro, leve, monitorável e resiliente. Esse checklist é uma espécie de guia prático para ajudar nesse momento decisivo.

1. Imagem otimizada: evite usar imagens genéricas e pesadas, como [node:latest](#). Prefira versões slim ou crie uma imagem multistage para garantir que apenas o essencial vá para produção;

2. Variáveis de ambiente seguros: verifique se as variáveis sensíveis (como senhas e tokens) estão sendo passadas via [secrets](#): e não expostas via [env_file](#) ou diretamente no `Compose`;

3. Limites de recursos definidos: defina limites claros de CPU e memória para evitar que o container sobrecarregue o host:

```
deploy:
  resources:
    limits:
      cpus: '0.50'
      memory: 512M
```

Comando de Prompt 5 – Definindo limites claros de CPU e memória
Fonte: elaborado pelo autor (2025)

4. Healthcheck configurado: um bom **healthcheck** ajuda o orquestrador a saber quando reiniciar o container. Ele também serve como alerta para instabilidades:

```
healthcheck:
test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
interval: 30s
  timeout: 10s
  retries: 3
```

Comando de Prompt 6 – Healthcheck bem configurado
Fonte: elaborado pelo autor (2025)

5. Log estruturado: certifique-se de que o app gera logs no formato JSON (ou outro formato estruturado) e que eles estão sendo enviados para uma stack de observabilidade (ex.: Fluentd, Loki, Promtail);

6. Usuário sem privilégios: evite rodar containers como root. Configure o Dockerfile ou Compose para usar um usuário sem privilégios:

```
USER node
```

Comando de Prompt 7 – Configurando o usuário sem privilégios
Fonte: elaborado pelo autor (2025)

7. Network isolada: use redes específicas para comunicação entre serviços, evitando o uso da **bridge** padrão. Isso aumenta a segurança e o controle;

8. Persistência de dados garantida: volumes bem configurados para garantir que nenhum dado importante será perdido após um restart;

9. Labels e metadados: inclua labels no Compose para facilitar a automação e a integração com ferramentas de observabilidade e monitoramento:

```
labels:
com.example.service: "minha-api"
```

Comando de Prompt 8 – Incluindo labels no Compose
Fonte: elaborado pelo autor (2025)

10. Teste final local: simule o ambiente de produção localmente usando `docker-compose -f docker-compose.yml -f docker-compose.prod.yml up`. Se estiver tudo certo, então pode subir com confiança.

Esse checklist não é só um ritual — é uma defesa contra surpresas desagradáveis. Com ele, você transforma seu container de algo que "funciona na minha máquina" para algo que roda bem, de forma previsível e segura, em produção.

EMANDA

MERCADO, CASES E TENDÊNCIAS

LIVRO: “Como se faz DevOps: Organizando pessoas, dos silos aos times de plataforma”

Autores: Leonardo Leite, Paulo Meirelles, Fabio Kon

Editora: Novatec

Ano: 2024

Para quem quer entender o DevOps além das ferramentas, este livro traz uma abordagem prática e crítica sobre como times reais se organizam para entregar software com qualidade e colaboração. Leia especialmente a **Parte 2**, que discute diferentes formas de estruturar equipes — desde departamentos isolados até modelos mediados por APIs — e como isso impacta a cultura, a eficiência e o alinhamento entre desenvolvimento e operações.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula final, você reuniu os aprendizados essenciais para aplicar Docker com mais maturidade em projetos reais. Começamos revisando as boas práticas, que ajudam a manter seus containers mais leves, organizados e eficientes. Depois, avançamos para os cuidados de segurança: desde como evitar armadilhas comuns até o uso de ferramentas para identificar vulnerabilidades. E, por fim, mergulhamos em técnicas de troubleshooting, aprendendo a identificar e resolver problemas de forma mais rápida e segura.

Agora você não só sabe construir e orquestrar containers, como também tem uma base sólida para manter ambientes saudáveis e estáveis no dia a dia. Esse é o tipo de conhecimento que faz a diferença na hora de trabalhar com aplicações em produção.

REFERÊNCIAS

HIGHTOWER, K.; BURNS, B.; BEDA, J. **Kubernetes: Up and Running**. Beijing: O'Reilly Media, 2017.

LEITE, L.; MEIRELLES, P.; KON, F. **Como se faz DevOps: organizando pessoas, dos silos aos times de plataforma**. São Paulo: Casa do Código, 2024.

TURNBULL, J. **The Docker Book: Containerization is the new virtualization**. 5. ed. USA: James Turnbull, 2014.

EMANIP

PALAVRAS-CHAVE

Containerização. Segurança da informação. Diagnóstico de falhas.

EMANIP



POS TECH

EMAP